Dissertation Implementation Plan

Jahangeer Khan · MSc Artificial Intelligence · SHU · Supervisor: Prof. Jing Wang

Project Overview

> *"Detecting Concept Drift in Non-IID Federated Learning Environments" Research Question: "Can cosine similarity detect concept drift in a Non-IID Federated Learning environment?"*

Problem	FL server cannot tell if a bad update is Non-IID (normal) or Concept Drift (needs action)
Solution	Cosine similarity of client's own gradient history detects sudden changes = drift
Baselines	FedAvg vs CDA-FedAvg vs Proposed Method
Datasets	CIFAR-10 (general) + UNSW-NB15 (IoT) + Credit Card Fraud (Finance)
Tools	Python, Flower (flwr), PyTorch, NumPy, Matplotlib, Scikit-learn

Literature Review — Key Papers to Read

Read abstract + conclusion only for each paper — do not read full paper at this stage!

Paper	What It Does	Where to Find	Why Read It
McMahan et al. (2017)	FedAvg — original FL paper	Google Scholar: 'FedAvg McMahan 2017'	Foundation — your baseline algorithm
Kairouz et al. (2021)	Open Problems in FL	Google Scholar: 'Open problems federated learning'	Confirms concept drift is unsolved
Casado et al. (2022)	CDA-FedAvg	Google Scholar: 'CDA-FedAvg concept drift'	Your direct baseline to beat
Mahdi et al. (2025)	FL Concept Drift Survey	mdpi.com/2079-9292/14/22/4480	Confirms your research gap
Ganguly & Aggarwal (2023)	Online FL Non-Stationary	arxiv.org/abs/2211.12578	Drift detection method reference
FL-MalDrift (2025)	Drift detection IoT/malware	nature.com/articles/s41598-025-31592-z	Real-world application example

■ *For each paper — write 3 sentences: What they did, What they missed, How it links to your research.*

Month 1 — May 2026: Setup + Literature + Basic FL

WEEK 1	1st May — 7th May	Literature Review + Setup

• Read McMahan 2017 (FedAvg) — abstract + conclusion only	PAPER
• Read Kairouz 2021 (Open Problems) — abstract + conclusion	PAPER
• Read Casado 2022 (CDA-FedAvg) — abstract + conclusion	PAPER
• Read Mahdi 2025 (Survey) — abstract + conclusion	PAPER
• For each paper: note what they did and what gap remains	NOTES
• Install Python virtual environment + all libraries	SETUP

```
# Activate environment fl_env\Scripts\activate # Verify installation python -c "import flwr; import torch; print('Ready!')"
```

WEEK 2	8th May — 14th May	Basic FL System

• Read Flower quickstart tutorial: flower.ai/docs/framework/tutorial-quickstart-pytorch.html	TUTORIAL
• Run Flower basic example with 5 clients and CIFAR-10	CODE
• Understand FedAvg aggregation code — how weights are averaged	CODE
• Plot accuracy graph over 20 rounds using matplotlib	CODE
• Submit proposal to Alice — 12th May deadline!	DEADLINE

```
# Basic FL structure # server.py — aggregates weights # client.py — trains locally # model.py — CNN model definition # Run: python server.py
```

WEEK 3	15th May — 21st May	Non-IID Simulation

• Implement Dirichlet distribution to split CIFAR-10 into Non-IID partitions	CODE

• Test with alpha=0.1 (severe), 0.5 (moderate), 1.0 (mild) Non-IID	EXPERIMENT
• Run FedAvg on Non-IID data — record accuracy drop vs IID	EXPERIMENT
• Plot IID vs Non-IID accuracy comparison graph	RESULTS
• Write notes: how much does Non-IID hurt FedAvg?	NOTES

```
# Dirichlet Non-IID split import numpy as np def dirichlet_split(dataset, n_clients, alpha): labels = np.array(dataset.targets) n_classes = len(set(labels)) # Split using Dirichlet distribution distribution = np.random.dirichlet([alpha]*n_clients, n_classes) return distribution
```

WEEK 4	22nd May — 31st May	Concept Drift Simulation

• Implement sudden concept drift — shift class labels at round 30	CODE
• Implement gradual drift — slowly shift distribution rounds 30-50	CODE
• Run FedAvg with drift — record accuracy drop at drift point	EXPERIMENT
• Plot accuracy curve showing drift impact	RESULTS
• Start writing Chapter 1: Introduction	WRITING

```
# Simulate concept drift at round 30 def apply_drift(data, round_num, drift_round=30): if round_num >= drift_round: # Shift labels — simulate drift data.targets = [(t + 5) % 10 for t in data.targets] return data
```

■ *Month 1 Goal: FedAvg running, Non-IID simulated, Drift simulated. You should have 3 graphs by end of May!*

Month 2 — June 2026: Cosine Similarity + Detection

WEEK 5	1st June — 7th June	Gradient History + Cosine Similarity

• Add gradient history storage to each client — save gradients after each round	CODE
• Implement cosine similarity calculation — current vs historical gradients	CODE
• Record cosine similarity scores for each client every round	CODE
• Plot cosine similarity over time for Non-IID clients	RESULTS
• Plot cosine similarity over time for drifting clients	RESULTS

```
# Cosine similarity implementation import torch import torch.nn.functional as F def compute_cosine_similarity(current_grad, historical_grad): # Flatten gradients curr = torch.cat([g.flatten() for g in current_grad]) hist = torch.cat([g.flatten() for g in historical_grad]) # Compute similarity similarity = F.cosine_similarity(curr.unsqueeze(0), hist.unsqueeze(0)) return similarity.item()
```

WEEK 6	8th June — 14th June	Threshold Finding

• Compare cosine similarity patterns: Non-IID (stable low) vs Drift (sudden drop)	EXPERIMENT
• Find threshold value that best separates Non-IID from Drift	EXPERIMENT
• Test different threshold values: 0.3, 0.4, 0.5 — measure detection accuracy	EXPERIMENT
• Calculate: precision, recall, F1 score of drift detection	RESULTS
• Write Chapter 2: Literature Review	WRITING

```
# Threshold-based drift detection def detect_drift(similarity_history, threshold=0.4, window=3): if len(similarity_history) < window + 1: return False recent_avg = sum(similarity_history[-window:]) / window previous_avg = sum(similarity_history[-(window*2):-window]) / window drop = previous_avg - recent_avg return drop > threshold # True = drift detected
```

WEEK 7	15th June — 21st June	Adaptive Weighting Mechanism

• Implement adaptive weight reduction for drifting clients	CODE
• When drift detected: reduce client contribution weight 100% to 30%	CODE
• Implement gradual weight restoration as client stabilises	CODE
• Run full experiment: FedAvg vs Your Method — compare accuracy	EXPERIMENT
• Record: accuracy, recovery speed, convergence stability	RESULTS

```
# Adaptive weight reduction def adaptive_aggregation(client_weights, drift_flags):
adjusted_weights = [] for i, (weight, is_drifting) in enumerate(zip(client_weights,
drift_flags)): if is_drifting: # Reduce drifting client contribution
adjusted_weights.append(weight * 0.3) else: adjusted_weights.append(weight * 1.0) #
Normalise weights total = sum(adjusted_weights) return [w/total for w in
adjusted_weights]
```

WEEK 8	22nd June — 30th June	CDA-FedAvg Baseline

• Implement CDA-FedAvg as second baseline (from Casado et al. 2022)	CODE
• Run CDA-FedAvg under Non-IID + Drift conditions	EXPERIMENT
• Compare all 3: FedAvg vs CDA-FedAvg vs Your Method	EXPERIMENT
• Build comparison table: accuracy under 4 conditions	RESULTS
• Write Chapter 3: Methodology	WRITING

■ *Month 2 Goal: Cosine similarity working, threshold found, all 3 methods compared. You should have full results table by end of June!*

Month 3 — July 2026: Multi-Domain Testing + Writing

WEEK 9	1st July — 7th July	IoT Domain Testing

• Download UNSW-NB15 dataset (IoT network intrusion — free)	SETUP
• Preprocess and split into Non-IID partitions across 5 clients	CODE
• Apply same cosine similarity method to IoT dataset	EXPERIMENT
• Compare results: does method work as well on IoT data?	RESULTS
• Document findings: applicability to IoT domain	WRITING

WEEK 10	8th July — 14th July	Finance Domain Testing

• Download Credit Card Fraud dataset from Kaggle (free)	SETUP
• Preprocess and split into Non-IID partitions across 5 clients	CODE
• Apply same cosine similarity method to Finance dataset	EXPERIMENT
• Compare results: does method work on Finance data?	RESULTS
• Write summary: domain-agnostic proof across 3 datasets	WRITING

WEEK 11	15th July — 21st July	Final Analysis + Graphs

• Produce final comparison graphs — all 3 methods, all 3 datasets	RESULTS
• Produce cosine similarity pattern graphs — Non-IID vs Drift	RESULTS
• Calculate statistical significance of improvements	RESULTS
• Write Chapter 4: Results and Analysis	WRITING
• Upload all code to GitHub repository	SETUP

WEEK 12	22nd July — 31st July	Evaluation + Stakeholder Feedback

• Create evaluation questionnaire for external stakeholders	EVALUATION
• Share results with Jing Wang — get expert feedback	EVALUATION
• Reach out to finance/IoT professionals via LinkedIn for feedback	EVALUATION
• Document all feedback received as evaluation evidence	EVALUATION
• Write Chapter 5: Discussion and Conclusion	WRITING

■ *Month 3 Goal: Solution tested on 3 domains, all graphs produced, external feedback collected, 4 chapters written!*

August 2026: Final Writing + Submission

WEEK 13-14	1st Aug — 14th Aug	Complete Dissertation Writing

• Finalise Chapter 1: Introduction	WRITE
• Finalise Chapter 2: Literature Review	WRITE
• Finalise Chapter 3: Methodology	WRITE
• Finalise Chapter 4: Results and Analysis	WRITE
• Finalise Chapter 5: Discussion and Conclusion	WRITE
• Write Abstract, References, Appendices	WRITE

WEEK 15-16	15th Aug — 31st Aug	Review + Submit

• Proofread entire dissertation	FINAL
• Check all references in APA format	FINAL
• Verify all graphs and tables are properly labelled	FINAL
• Get Jing to review final draft	FINAL
• Submit dissertation!	DEADLINE

Dissertation Structure

Chapter	Title	Content
Chapter 1	Introduction	FL background, problem statement, research question, dissertation overview
Chapter 2	Literature Review	FL algorithms, Non-IID problem, Concept Drift in FL, cosine similarity usage, research
Chapter 3	Methodology	Experimental design, datasets, FL setup, drift simulation, cosine similarity implementa
Chapter 4	Results & Analysis	Comparison tables, graphs, threshold analysis, domain-agnostic results
Chapter 5	Discussion & Conclusion	Findings, real-world applicability, limitations, future work (XAI, decentralised FL)
Appendices	Supporting Material	Ethics form, code snippets, stakeholder feedback, raw data tables